

Scaling the Construction-VLM XAI Evaluation: Phase 1 — Foundational Correctness Fixes

Engineering log, prepared for research collaboration with Professor Mustafa Abdallah

July 9, 2026

Contents

1 Rule-Aware Region Ranking (Phase 1.1)	4
1.1 Goal	4
1.2 What Was Done	4
1.2.1 A ranking policy, selected by flag, defaulting to the frozen behavior	4
1.2.2 One source of truth for the object-class map	5
1.2.3 Configuration and record protection	5
1.2.4 Test-first	5
1.3 Results	5
1.3.1 The ranking now selects the right entity	5
1.3.2 The 11.9-point gap closes	6
1.3.3 The frozen record is intact	6
1.4 A Problem This Surfaced, and Its Resolution	6
1.5 Standard vs. Non-Standard Choices	7
1.6 Implications	7
2 Multi-Label Classification and Class-Balanced Sampling (Phase 1.2)	8
2.1 Goal	8
2.2 What Was Done	8
2.2.1 Exposing the full violated-class set	8
2.2.2 A labeling-aware sampler	8
2.2.3 Manifest and configuration	9
2.3 Results	9
2.4 A Scoping Decision, Stated Plainly	9
2.5 Standard vs. Non-Standard Choices	10
2.6 Implications	10

3	Compliant-Sample Context Matching (Phase 1.3)	11
3.1	Goal	11
3.2	What Was Done	11
3.2.1	The one reliable metadata context signal	11
3.2.2	The context-matched assignment policy	12
3.3	Results	12
3.4	A Limitation, Stated Honestly	13
3.5	Standard vs. Non-Standard Choices	13
3.6	Implications	13
4	Worker-Loss Instrumentation Baked Into the Metrics (Phase 1.4)	14
4.1	Goal	14
4.2	What Was Done	14
4.2.1	A worker-loss flag in every affected metric	14
4.2.2	Reporting, gated and record-safe	15
4.3	Results	15
4.3.1	Descriptive accuracy: raw vs. genuine	15
4.3.2	Robustness: occlusion is where worker loss bites	15
4.3.3	Completeness: the 159 reruns, replaced by a rollup	16
4.4	A Subtlety Worth Recording	16
4.5	Standard vs. Non-Standard Choices	16
4.6	Implications	16
5	Prompt Dictionary Concept Split (Phase 1.5)	18
5.1	Goal	18
5.2	What Was Done	18
5.2.1	A concept/synonym structure	18
5.2.2	Frozen queries preserved	19
5.3	Results	19
5.4	An Honest Note on Synonym Strength	20
5.5	Standard vs. Non-Standard Choices	20
5.6	Implications	20
6	Unifying Decoding to a Single Greedy Path (Phase 1.6)	21

6.1	Goal	21
6.2	What Was Done	21
6.3	Results	21
6.4	Standard vs. Non-Standard Choices	22
6.5	Implications	22
7	Hardcoded-Assumption Audit and a CI Guard (Phase 1.7)	24
7.1	Goal	24
7.2	What Was Done	24
7.2.1	The patch-grid arithmetic, now derived	24
7.2.2	The remaining label literals	24
7.2.3	A CI guard that tells code from documentation	25
7.3	Results	25
7.4	Standard vs. Non-Standard Choices	25
7.5	Implications	25

Chapter 1

Rule-Aware Region Ranking (Phase 1.1)

1.1 Goal

The pilot’s single highest-leverage defect, identified in Chapter 10 of the pilot report and restated as item 3.1 of the scale-up plan, is that candidate explanation regions were ranked by pixel *area*. Florence-2 returns one box for the worker and one box for the queried safety object (a hard hat, a harness, a guardrail, an excavator); the worker’s whole-body box is almost always larger than a hard hat or a harness worn on that body, so area ranking put the *worker* at rank 1 in 96% of PPE (rule_1) samples. Every downstream masking and overlap metric then perturbed or measured the worker box instead of the safety object the rule is actually about.

The pilot report converted this observation into a falsifiable prediction, which is the target this phase must hit. Splitting the 163 pilot samples by which entity was ranked top-1, masking the worker flipped the answer in only 37.5% of cases versus 49.4% for the actual safety object — an **11.9 percentage-point gap** that the report attributed entirely to the ranking heuristic promoting the wrong entity, not to any property of the model. The prediction: once ranking stops systematically promoting the worker, that gap should “narrow substantially, if not close outright.”

1.2 What Was Done

1.2.1 A ranking policy, selected by flag, defaulting to the frozen behavior

`regions.standardize_regions` gained a `region_ranking` parameter taking one of three values:

- `"area"` — the frozen pilot behavior: model boxes sorted by descending area. This remains the default so that every recorded pilot number reproduces byte-for-byte.
- `"rule_aware"` — regions whose label is the rule’s queried safety object are promoted above all others; within each tier, boxes are still ordered by descending area. This is the scale-up policy.
- `"attention"` — reserved for Phase 4, where cross-attention / Integrated-Gradients mass will supply a real per-region importance signal. Until that signal exists, the mode raises `NotImplementedError` rather than silently falling back to area, so that no run can quietly produce area-ranked numbers while believing they are attention-ranked.

The two-tier sort is implemented as two stable sorts: first by area (descending), then by a tier key (0 if the region’s label is the rule’s object, 1 otherwise). Python’s stable sort preserves the area ordering *inside* each tier, so the result is exactly “objects first, largest object first; then everything else, largest first.” `rule_aware` requires a non-empty set of object labels and raises `ValueError` if none is supplied,

again refusing to degrade silently.

1.2.2 One source of truth for the object-class map

The rule→object-label map ({rule_1: hard hat, rule_2: harness, rule_3: guardrail, rule_4: excavator}) had been copy-pasted inline into three separate metric scripts (04_extract_regions, 05_descriptive_accuracy, 07_stability_test). It now lives once in `prompts.RULE_OBJECT_LABEL`, with a helper `rule_object_labels(rule_id)` returning the object-label set that `rule_aware` ranking consumes. The three scripts import the shared map instead of redeclaring it — eliminating a triplicated literal that was itself a latent instance of the hardcoded-assumption class of bug Phase 1.7 targets.

1.2.3 Configuration and record protection

`config.REGION_RANKING` holds the default policy ("area"). Scripts 04, 05, and 07 each gained a `--region-ranking` command-line flag defaulting to that config value. Critically, any non-default run writes to a *mode-suffixed* output file (`descriptive_accuracy_rule_aware.csv`, not `descriptive_accuracy.csv`), so a scale-up validation run can never overwrite the frozen pilot record it is being compared against.

1.2.4 Test-first

Seven unit tests were written for the new behavior and watched to fail before any implementation existed: object-over-larger-worker promotion, area ordering within each tier, grid fallback under `rule_aware`, the empty-object_labels guard, the unknown-mode guard, and the `attention` not-yet-implemented guard, plus a test pinning the default to area ranking.

1.3 Results

1.3.1 The ranking now selects the right entity

Re-ranking the 163 pilot samples' already-extracted boxes (no model re-inference needed for this part) shows the policy doing exactly what it should. Among the 159 samples that had a model box at all (four fell back to the grid under both policies):

Top-1 region is...	area (frozen)	rule_aware
the worker body	50.3%	0.6%
the rule's queried object (rule_1/2/3)	—	100%
the rule's queried object (rule_4)	—	96%

Table 1.1: Which entity the ranking promotes to rank 1, across the 159 model-box pilot samples. Under area ranking the worker body was top-1 in just over half of all samples; under `rule_aware` it is top-1 in one.

For PPE (rule_1) specifically — the case the pilot cares most about, and the case the old ranking handled worst — the top region changed in 92.1% of samples. It changed far less for rule_3 (6.5%) and rule_4 (0%), which is correct behavior, not a weakness: guardrails and excavators are often already the

largest box in frame, so area ranking and rule-aware ranking agree on them. The fix moves precisely the samples that needed moving.

1.3.2 The 11.9-point gap closes

Reproducing the pilot report’s worker-vs-object split on the actual re-run descriptive-accuracy results:

	area (frozen)	rule_aware
top-1 = worker: flip rate	30.0% ($n=80$)	0.0% ($n=1$)
top-1 = object: flip rate	43.0% ($n=79$)	39.2% ($n=158$)
worker-vs-object gap	+13.0 pp	eliminated (no worker group left)

Table 1.2: The worker-vs-object descriptive-accuracy gap, before and after. Under area ranking, 80 of 159 samples masked the worker; under rule_aware, one does. The systematic gap is gone because the confound that produced it — the ranking’s inconsistency about which entity it promotes — is gone.

Two honest notes on these numbers. First, the frozen gap reproduces here as 13.0 pp using the raw top-1 answer-flip signal, versus the report’s 11.9 pp, which was computed on the bounded-completeness “supported” verdict (a signal that also folds in the top-2 mask). It is the same phenomenon measured one step upstream; the magnitude agrees to within a point. Second, the object-group flip rate drops slightly, from 43.0% to 39.2%, once ranking is fixed. This is not a regression: the 79 samples that previously masked the worker (mostly PPE) now join the object group, and their genuine object-mask flip rate is lower than the rate of the samples that already had a large object on top. The fix does not inflate the metric — it de-contaminates it, and de-contamination reveals that masking a small PPE object flips the proxy less often than the old, worker-contaminated aggregate suggested. PPE’s own descriptive accuracy nonetheless rises from 27.0% to 33.3% now that its top-1 mask lands on the hard hat rather than the person.

1.3.3 The frozen record is intact

Running `04_extract_regions.py --region-ranking area` regenerated `region_extraction.csv` byte-for-byte identical to the committed pilot file (empty `git diff`), confirming the refactor did not disturb the default path. The full unit suite is green at 68 tests (61 original plus the 7 added here).

1.4 A Problem This Surfaced, and Its Resolution

Masking metrics carry a known artifact from the pilot: masking the *worker* box can make the worker undetectable on the rerun, which reroutes `_answer_presence_rule` into a degenerate scene-level fallback and flips the answer for a reason unrelated to the safety concept. The pilot measured this “worker-loss” signature as 15 samples where masking top-1-plus-top-2 *un-flips* an answer that masking top-1 alone had flipped.

After rule-aware ranking, that count rose to 22, not fell. This looks alarming until traced: it is the same artifact *relocating*, not worsening. Under area ranking the worker was top-1, so worker-loss struck at the top-1 mask; under rule-aware ranking the worker is now top-2, so worker-loss strikes at the top-2

mask instead. The number of contaminated samples is similar; the stage at which the contamination appears has moved because the worker moved down the ranking.

The resolution is architectural, not a patch to this phase: Phase 1.1’s job is to make the metric test the *right box*, and it does. De-contaminating the worker-loss fallback is a separate, explicitly-scoped item (Phase 1.4, worker-loss instrumentation), which will record post-mask worker boxes and report each metric twice — raw and worker-loss-corrected. This phase’s result should therefore be read as “the ranking is now correct,” and the aggregate descriptive-accuracy figure should not be treated as trustworthy until Phase 1.4 lands. Keeping the two fixes as distinct items, rather than entangling them, is what makes each independently verifiable.

1.5 Standard vs. Non-Standard Choices

Standard. Ranking explanation regions by semantic relevance to the query, rather than by raw size, is ordinary practice; area was only ever a stopgap the pilot adopted because Florence-2’s grounding path exposes no native importance score.

Non-standard, and the actual discipline. Three choices go beyond the obvious fix. The old behavior is preserved exactly, behind a default-valued flag, so the pilot record remains a reproducible baseline rather than being overwritten by the improvement. Non-default runs are physically prevented from clobbering that record by output-file suffixing. And the not-yet-built **attention** mode fails loudly instead of silently falling back to area — encoding the principle that a metric must never quietly report a different computation than the one its configuration claims. These are the habits that let a scale-up change a pipeline’s behavior without losing the ability to prove what the old behavior was.

1.6 Implications

Rule-aware ranking unblocks every masking and overlap metric (descriptive accuracy, sparsity, stability, completeness) to operate on the safety object rather than the worker body, and it does so with no additional model calls — purely a re-ordering of boxes Florence-2 has already returned. The pilot report’s falsifiable prediction is met: the 11.9-point worker-vs-object gap closes outright. The immediate caveat carried forward is that the worker-loss fallback, now relocated to the top-2 stage, must be instrumented (Phase 1.4) before the corrected descriptive-accuracy, robustness, and completeness aggregates can be quoted.

Chapter 2

Multi-Label Classification and Class-Balanced Sampling (Phase 1.2)

2.1 Goal

The pilot assigned each image exactly one class by *priority collapse*: a row that violated more than one rule was labeled only with its highest-priority violation (`ppe_violation > fall_hazard > struck_by_risk`), and every lower-priority violation on the same image was discarded. This is a labeling convention, not a property of the dataset, and it has a concrete, damaging consequence for the scarcest and most safety-critical class. An image showing a worker with no hard hat *and* standing in an excavator’s swing radius is a real struck-by hazard, but priority collapse files it as a PPE violation and the struck-by label is lost. The pilot’s `struck_by_risk` class was therefore powered at only $n = 13$ in the entire test split — too small for any stable estimate.

The scale-up plan (item 3.2) diagnoses this precisely: the `struck_by_risk` scarcity is “a labeling artifact, not a dataset property,” and predicts that multi-label classification recovers the hidden examples. The goal of this phase is to fix the labeling policy so that an image counts toward — and is tested against — every rule it actually violates, and to verify the predicted recovery.

2.2 What Was Done

2.2.1 Exposing the full violated-class set

The information priority collapse discards was already being computed and thrown away. A new function, `data.image_classes(row)`, returns the *complete* list of violated classes for a row (highest-priority first), where the existing `classify_image` returns only the single collapsed primary class. A clean row returns `["compliant"]`; a `rule_1+rule_4` row returns `["ppe_violation", "struck_by_risk"]`; a `rule_2+rule_3` row returns `["fall_hazard"]` once, since both those rules map to the same class.

2.2.2 A labeling-aware sampler

`select_balanced_sample` gained a `labeling` parameter:

- **"priority"** (default): the frozen pilot sampler, untouched in behavior — one class and one rule per image. Its only addition is that it now also populates the new `assigned_rule_ids` field with the single-element list `[assigned_rule_id]`, so that both labeling modes present the same interface downstream.
- **"multilabel"**: an image is placed into the candidate reservoir of *every* class it violates, so a `rule_1+rule_4` image sits in both the `ppe_violation` and `struck_by_risk` pools and can be drawn to satisfy either class’s coverage floor. After per-class selection, images drawn via more than one

class are deduplicated by `image_id` and their violated rules unioned, so each unique image appears once carrying *all* the rules it is tested against.

The `PilotSample` dataclass gained an `assigned_rule_ids: list[str]` field to hold that per-image rule set. Determinism is preserved exactly: multi-label selection uses the same seeded shuffle in the same class order as the priority sampler, and deduplication preserves first-encounter order.

2.2.3 Manifest and configuration

`config.LABELING` holds the default policy ("priority"). Script `02_select_samples.py` gained a `--labeling` flag. Under priority it writes the original five-column `pilot_samples.csv` unchanged; under multi-label it writes an expanded six-column, mode-suffixed manifest (`pilot_samples_multilabel.csv`) with an `assigned_rule_ids` column, so the two policies never share an output file.

2.3 Results

Scanning the full 3,004-image test split (reading only the four rule columns from the local parquet, no image decoding), the predicted recovery appears exactly as the plan described it:

class	priority (primary)	multi-label (coverage)
ppe_violation	324	324
fall_hazard	75	86
struck_by_risk	13	24
compliant	2,592	2,592

Table 2.1: Per-class counts on the full test split. Priority-collapse counts each image once, under its highest-priority class; multi-label counts an image toward every class it violates.

The `struck_by_risk` pool nearly doubles, from 13 to 24 — a recovery of exactly 11 images. Those 11 are precisely the images that violate `rule_4` while *also* carrying a higher-priority violation that priority-collapse had used to file them elsewhere; the count was verified directly, not inferred: 11 struck-by images have a non-struck-by primary class under the old policy. This reproduces to the image the pilot report’s own statement that “11 of 24 real struck_by_risk test-split images also had a higher-priority violation.” `fall_hazard` likewise recovers 11 images (75 → 86) hidden behind PPE violations. `ppe_violation` and `compliant` are unchanged, as expected: PPE is the top priority (nothing outranks it to hide it) and `compliant` images have no violations to redistribute.

The full unit suite is green at 76 tests (68 after Phase 1.1, plus 7 new multi-label tests written test-first, plus one already counted). The seven new tests pin the full-set classification, the `fall_hazard` de-duplication, the per-image multi-rule assignment, the struck-by recovery, and the guarantee that a shared image is never duplicated across class reservoirs.

2.4 A Scoping Decision, Stated Plainly

This phase deliberately fixes the *data layer* — classification, sampling, and per-image rule assignment — and stops there. It does *not* rewire the downstream metric scripts (baseline inference through

completeness) to iterate over (`image`, `rule`) test units rather than one row per image. That rewiring is real work with real blast radius: every metric script currently keys on `image_id` with a single assigned rule, and `baseline_predictions.csv` is one row per image.

The decision to defer it is not a shortcut; it follows the plan’s own structure. The scale-up plan couples the multi-label fix directly with the Phase 5 scale run (“scale from 163 to the full 3,004-image test split ... pair with the multi-label fix so `struck_by_risk` is powered”). Doing the end-to-end rewiring now would fork the frozen 163-sample set in the middle of the Phase 1 validation sequence, where every other fix is being checked *against* that same frozen set. Building the recovery mechanism now, and consuming it at scale later, keeps each Phase 1 fix independently verifiable against a stable baseline. The consumption path is a Phase 5 deliverable, and is called out as such here so the boundary is explicit rather than implied.

2.5 Standard vs. Non-Standard Choices

Standard. Multi-label treatment of images that genuinely exhibit multiple hazards is the correct and conventional framing for a multi-hazard detection problem; single-label collapse was only ever a simplification the pilot adopted to get a balanced four-class subset quickly.

Non-standard, and worth noting. The recovery was validated by reading only the rule columns from the parquet and counting, rather than by running the full sampler over decoded images — a deliberately cheap check that isolates the one claim being tested (does multi-label surface the hidden struck-by images?) from every slower, unrelated concern. And, consistent with Phase 1.1, the new policy is gated behind a default-valued flag that writes to a separate manifest file, so the frozen pilot manifest and every number derived from it remain reproducible while the multi-label manifest exists alongside it.

2.6 Implications

`struck_by_risk` is no longer inherited at $n = 13$; on the test split alone it is powered at $n = 24$, and across the train split (available for supplementing minority classes) the plan’s target of roughly 70 usable examples is now reachable. More importantly, multi-label data is the precondition for the multi-hazard completeness tests the proposal promised (Phase 2.5): an image that violates two rules can now be labeled, sampled, and — once the Phase 5 consumption path lands — tested for whether the explanation captures *both* hazards. The immediate carry-forward is that boundary: the sampler and manifest speak multi-label now, and the metric scripts will learn to consume it during the scale run.

Chapter 3

Compliant-Sample Context Matching (Phase 1.3)

3.1 Goal

Compliant images carry no violated rule, so the pilot assigned each one a rule to be tested against by plain round-robin: the first compliant image was tested on `rule_1`, the next on `rule_2`, and so on, cycling through all four rules regardless of what the image actually contained. This produces a balanced “model correctly says compliant” coverage across rules, but it also produces *vacuous* tests. A compliant image of flat ground with no excavator anywhere in frame, round-robin to `rule_4` (worker-in-excavator-swing-radius), tests nothing: there is no hazard context for the model to correctly clear. Worse, the vacuous pass counts as a correct “compliant” answer and silently inflates baseline compliant accuracy.

The scale-up plan (item 3.3) prescribes the fix: when assigning a rule to a compliant image, prefer a rule whose *context is present* in the scene, using dataset metadata where available and marking the assignment `context_absent` where no contextual match can be confirmed, so those cases are reported separately rather than folded into the accuracy number.

3.2 What Was Done

3.2.1 The one reliable metadata context signal

The ConstructionSite parquet carries per-image metadata columns beyond the rule labels: `illumination`, `camera_distance`, `view`, `quality_of_info`, and three object-box columns — `excavator`, `rebar`, and `worker_with_white_hard_hat`. Inspecting their values across the test split establishes what can and cannot be trusted as a rule context signal:

- `excavator` is a list of bounding boxes, non-empty in exactly the images that contain an excavator. This is a high-precision, direct signal for `rule_4`’s struck-by context.
- `view` is “elevation view” (2,895) vs “plan view” (109) — a weak, necessary-not-sufficient hint at height, not a reliable `rule_2`/`rule_3` context signal.
- `worker_with_white_hard_hat` only marks workers wearing a *white* hard hat, so its emptiness does not imply “no worker”; it is not a trustworthy `rule_1` (PPE) context signal.
- `illumination`, `camera_distance`, `quality_of_info` describe capture conditions, not rule context.

The honest conclusion drives the design: `excavator` is the only rule-context signal reliably derivable from metadata alone. Confirming PPE (`rule_1`), height (`rule_2`), or edge (`rule_3`) context requires actually looking at the image — a lightweight grounding pre-check — which is GPU work deferred to

a later phase.

3.2.2 The context-matched assignment policy

`assign_compliant_rule(has_excavator, index, mode)` returns `(rule_id, context_absent)`:

- "round_robin" (default, frozen): cycle all four rules; `context_absent` is always False because context is not assessed. Byte-for-byte identical to the pilot.
- "context_matched" (Phase 1.3): if the scene has an excavator, assign rule_4 — its struck-by context is confirmed present. If it does not, *never* assign the vacuous rule_4; round-robin over rules 1–3 instead and flag the assignment `context_absent=True`, recording honestly that metadata cannot confirm that rule’s context and a grounding pre-check would be needed.

A `context_absent` field was added to `PilotSample`, and `config.COMPLIANT_ASSIGNMENT` holds the default policy. Both the priority and multi-label samplers were refactored to share one assignment routine, capturing excavator presence during the streaming pass only when context matching actually needs it, so the frozen round-robin path does no extra per-row work and reproduces exactly.

3.3 Results

Applied to all 2,592 compliant images in the test split (of which 915, or 35.3%, contain an excavator):

	round-robin	context-matched
rule_4 (struck-by) assignments	648	915
of which vacuous (no excavator)	413 (63.7%)	0
excavator scenes not tested for struck-by	680	0
assignments flagged <code>context_absent</code>	0 (not assessed)	1,677 (64.7%)

Table 3.1: Compliant-image rule assignment on the full test split. Round-robin assigns rule_4 to images without any excavator (a vacuous struck-by test) and, conversely, fails to test most excavator scenes for struck-by risk at all. Context matching eliminates both.

Under round-robin, of the 648 compliant images assigned the struck-by rule, 413 (63.7%) had no excavator in frame — the test could only ever return “compliant” for lack of any hazard, and those 413 passes were quietly padding the compliant accuracy. Simultaneously, 680 images that *did* contain an excavator were round-robined onto some other rule, so their genuine struck-by context went untested. Context matching drives both failure modes to zero: every one of the 915 excavator scenes is now tested for struck-by risk, and not one excavator-free scene is. The 1,677 remaining compliant images (64.7%) are flagged `context_absent` rather than silently trusted — an explicit statement that their rule_1/2/3 context is unconfirmed by metadata and awaits the grounding pre-check.

The full unit suite is green at 83 tests (76 after Phase 1.2 plus 7 new context-matching tests written test-first), covering the excavator signal, both assignment modes, the no-vacuous-rule_4 guarantee, and end-to-end sampling under context matching.

3.4 A Limitation, Stated Honestly

Context matching only confidently confirms rule_4 context, because **excavator** is the only trustworthy context signal in the metadata. Rule_1 (is a worker present?), rule_2 (is anyone working at height?), and rule_3 (is there an unguarded edge?) cannot be confirmed from metadata, so excavator-free compliant images are flagged **context_absent** rather than context-matched. This is not a gap papered over — it is surfaced as an explicit count (64.7% of compliant images) and routed to a named future step: a lightweight grounding pre-check that detects worker/edge/height cues before assigning rules 1–3. That pre-check is GPU work coupled to the attribution and scale phases, and was deliberately not built here to keep Phase 1 free of new model calls. The value already delivered stands on its own: the single most clearly vacuous case, struck-by tests on excavator-free scenes, is eliminated, and the most clearly missed case, excavator scenes never tested for struck-by, is recovered.

3.5 Standard vs. Non-Standard Choices

Standard. Conditioning a test on whether its precondition is present in the input is ordinary experimental hygiene; a test with no stimulus measures nothing.

Non-standard, and the actual discipline. Two choices matter. First, refusing to over-claim: rather than inventing weak proxies for worker/height/edge context and pretending they are reliable, the policy confirms only what the metadata actually supports (excavator) and marks everything else **context_absent** for separate reporting. A vacuous test mislabeled as context-present would be worse than an honestly-flagged one. Second, and consistent with the whole of Phase 1, the fix sits behind a default-valued flag that leaves the frozen round-robin path bit-identical, so the pilot’s baseline compliant accuracy — inflated as it was by the 413 vacuous passes — remains reproducible as the baseline the corrected number is measured against.

3.6 Implications

Baseline compliant accuracy can now be reported two ways: the raw round-robin number (with its 413 vacuous struck-by passes) and a context-matched number that tests each compliant image only on rules whose context is present. The struck-by class, already recovered on the violation side by Phase 1.2, now also gets its compliant counterpart tested wherever an excavator actually appears — 915 scenes instead of an accidental 235. The carry-forward is the grounding pre-check for rules 1–3, which will convert the 64.7% **context_absent** compliant pool from “unconfirmed” into “context-matched or genuinely absent,” completing what metadata alone cannot finish.

Chapter 4

Worker-Loss Instrumentation Baked Into the Metrics (Phase 1.4)

4.1 Goal

Three of the six pilot metrics — descriptive accuracy, robustness, and bounded completeness — work by re-running the safety proxy on a modified image (a mask or a perturbation) and checking whether the answer flips. The pilot discovered, and documented across Chapters 5, 9, and 10, that a specific artifact contaminates all three: when the modification makes the *worker* undetectable, the proxy falls out of its per-worker check (`if not worker_boxes`) into a cruder scene-level fallback, and the answer can flip for a reason that has nothing to do with the safety concept being tested. A flip caused this way is a false signal of “the region/answer was sensitive to the safety cue.”

The pilot handled this reactively and expensively. Chapter 10, in particular, had to run 159 fresh top-1-mask GPU reruns *solely* to recover the post-mask worker boxes that Chapter 5’s descriptive CSV never logged, just to estimate how many “supported” completeness verdicts were really worker-loss artifacts. The scale-up plan (item 3.4) prescribes making this a standard, built-in part of every affected metric’s output: record post-operation worker boxes, emit a `flip_due_to_worker_loss` flag, and report each metric twice — **raw** (all flips) and **genuine** (worker-loss flips excluded).

4.2 What Was Done

4.2.1 A worker-loss flag in every affected metric

Each metric’s pure comparison core now derives the flag from information it already had access to but was discarding:

- **Descriptive accuracy** gained a pure core, `_descriptive_from_results`, that computes `worker_lost_top1/top2` (baseline had a worker, masked rerun has none) and `flip_due_to_worker_loss_top1/top2` (an answer flip that co-occurs with worker loss). Extracting the pure core also made the logic unit-testable without a GPU, which it previously was not.
- **Robustness** already exposed `worker_lost`; it gained the derived `flip_due_to_worker_loss = answer_changed AND worker_lost`, so the correction is a first-class metric field rather than something each analysis must recompute.
- **Completeness** gained `classify_sample_worker_loss_corrected`, which counts a sample “supported” only if a *genuine* flip (not attributable to worker loss) remains at top-1 or top-2.

Twelve unit tests were written test-first across the three metrics, covering the flip/no-flip × worker-lost/worker-present matrix and the corrected-verdict downgrade logic.

4.2.2 Reporting, gated and record-safe

A `config.REPORT_WORKER_LOSS_CORRECTED` flag (default `False`) threads through scripts 05, 09, and 10 as `--report-worker-loss-corrected`. Default off reproduces the frozen metric CSVs byte-for-byte; on, each script logs the worker-loss columns, reports raw and genuine rates side by side, and writes to a `_wlc`-suffixed file so no frozen artifact is overwritten.

The most consequential structural change is in completeness. Because descriptive accuracy now logs the post-mask worker boxes itself, the bounded-completeness verdict becomes a *pure rollup* of the `_wlc` descriptive CSV — the 159 fresh GPU reruns Chapter 10 needed are simply gone, replaced by reading two columns that already exist.

4.3 Results

4.3.1 Descriptive accuracy: raw vs. genuine

ranking	top-1 raw	top-1 genuine	worker-loss flips (top-1)
area (frozen)	36.2%	34.4%	3 of 59
rule_aware	38.7%	37.4%	2 of 63

Table 4.1: Descriptive accuracy, raw vs. worker-loss-corrected. The area raw number reproduces the frozen pilot’s 36.2% exactly. Under `rule_aware` ranking the top-1 worker-loss count falls ($3 \rightarrow 2$) because the worker is no longer the top-1 masked region — confirming, from the opposite direction, the Phase 1.1 observation that the artifact relocates to the top-2 stage under the fixed ranking (top-2 genuine is 31.3% under both rankings).

4.3.2 Robustness: occlusion is where worker loss bites

Computed directly from the frozen `robustness.csv` (which already carried `answer_changed` and `worker_lost`), the correction is negligible for most perturbations and material for exactly the one that physically covers the worker:

perturbation	raw	genuine	worker-loss flips
occlude	28.2%	22.7%	9 of 163
low_light	11.7%	11.0%	1 of 163
blur	27.0%	27.0%	0
contrast_shift	10.4%	10.4%	0
reworded_prompt	27.5%	27.5%	0
overall	20.8%	19.5%	10 of 790

Table 4.2: Robustness answer-change rate, raw vs. genuine. Occlusion’s 5.5-point inflation is concentrated worker loss — the centered occlusion square covers the worker and reroutes the proxy — directly supporting the plan’s separate call (Phase 2.4) to split targeted from random occlusion.

4.3.3 Completeness: the 159 reruns, replaced by a rollout

ranking	supported (raw)	supported (cor- rected)	downgraded
area (frozen)	71/163 (43.6%)	67/163 (41.1%)	4
rule_aware	78/163 (47.9%)	75/163 (46.0%)	3

Table 4.3: Bounded-completeness supported rate, raw vs. worker-loss-corrected, as a pure rollout of the `_wlc` descriptive CSV.

The area raw number, 71/163 (43.6%), reproduces Chapter 10’s headline exactly. The corrected number, 67/163 (41.1%), lands within one sample of Chapter 10’s own estimate of 68/163 (41.7%) — and where Chapter 10 spent 159 fresh GPU reruns to produce that estimate, this reproduces it with **zero** reruns, reading columns the descriptive metric now logs by default. The one-sample difference is explained, not hand-waved: Chapter 10’s estimate corrected only top-1 worker loss, whereas the baked-in verdict discounts worker-loss flips at both top-1 and top-2, so it catches one additional artifact.

4.4 A Subtlety Worth Recording

The correction is deliberately conservative about *which* flips it discounts. A flip is removed only when the answer changed *and* the worker was lost on that exact rerun — not whenever a worker was merely lost. A worker can vanish without the answer changing (the scene-level fallback happens to return the same answer), and that case is correctly *not* counted as a worker-loss flip, because nothing about the reported metric was corrupted. Conversely, a genuine flip where the worker is still detected is never discounted. This keeps “genuine” an honest lower bound on true sensitivity rather than an over-aggressive scrub that would hide real signal.

4.5 Standard vs. Non-Standard Choices

Standard. Reporting a corrected metric alongside a raw one, with the correction’s mechanism stated, is ordinary rigor once a confound is known.

Non-standard, and the real contribution. Two things. First, the correction is promoted from a one-off, expensive audit (Chapter 10’s 159 reruns) into standing metric output computed for free from data the metric now logs — the artifact is measured every run, not estimated once. Second, and in keeping with the whole phase, the raw metric is preserved bit-for-bit behind the default flag, so the pilot’s published raw numbers remain the reproducible baseline against which the genuine numbers are read; the correction adds a column, it never silently rewrites the number a prior chapter reported.

4.6 Implications

With Phases 1.1 and 1.4 together, the masking and perturbation metrics now both test the right region *and* report a worker-loss-corrected value, so their aggregates can finally be quoted as measuring the safety concept rather than the proxy’s fallback plumbing. The robustness result sharpens a specific Phase 2 task: occlusion’s inflation is worker loss, which is why targeted and random occlusion must be separated and reported apart. And completeness is now cheap enough to recompute on every scale run,

because its worker-loss correction no longer carries a 159-rerun tax.

Chapter 5

Prompt Dictionary Concept Split (Phase 1.5)

5.1 Goal

Florence-2 has no free-form VQA token, so every safety rule is answered by grounding a short noun phrase. The pilot stored two phrases per rule in `RULE_QUERIES` and used index 0 for the answer and index 1 for the Day-9 “rewording-robustness” test, which reruns the rule with the second phrasing and checks whether the answer or grounded box changes. The premise of that test is that index 0 and index 1 are *synonyms* — two ways of saying the same thing — so any change measures sensitivity to *wording*.

That premise is false for two of the three presence rules. For `rule_1` the two phrases are “hard hat” and “high-visibility vest”; for `rule_2` they are “safety harness” and “fall-protection lanyard.” These are not synonyms — they are *distinct physical objects*. Only `rule_3`’s pair, “guardrail” and “edge protection barrier,” are genuine synonyms of one object. So the pilot’s rewording test, for rules 1 and 2, was silently measuring how the answer changes when you ask about a *different object*, and calling that “robustness to rewording.” The plan (item 3.5) prescribes restructuring the query dictionary so each distinct object is its own concept with its own synonym list, and so the rewording test only ever swaps validated synonyms within a concept.

5.2 What Was Done

5.2.1 A concept/synonym structure

A new `RULE_CONCEPTS` maps each rule to {concept: [synonyms]}, disentangling the two things the flat `RULE_QUERIES` conflated:

rule	concepts (distinct objects)	synonyms per concept
rule_1	hard hat; high-visibility vest	{hard hat, safety helmet}; {hi-vis vest, ...}
rule_2	safety harness; fall-protection lanyard	{safety harness, fall-arrest harness}; ...
rule_3	guardrail (one concept)	{guardrail, edge protection barrier}

Table 5.1: The reviewed concept structure. rule_1 and rule_2 have two distinct concepts each; rule_3 has one concept with two genuine synonyms – the only rule whose Day-9 “rewording” was actually a rewording.

A companion `RULE_SATISFACTION` records whether a rule needs *all* its concepts present or *any* one (rule_1 “all”: both hard hat and vest are expected PPE; rule_2/rule_3 “any”), with an explicit note that the pilot proxy approximated rule_1 using only the hard-hat concept, so its operative semantics were narrower than the recorded “all” intent. Helper functions expose the structure (`concepts_for`, `synonyms_for`, `is_synonym`), and `reworded_query` returns a validated same-concept synonym for the corrected rewording test — never a different concept.

5.2.2 Frozen queries preserved

`RULE_QUERIES` is kept byte-for-byte (its index 0/index 1 phrases are unchanged), so every existing script and the frozen Day-9 results reproduce exactly. The concept structure is additive: it sits beside the frozen dictionary and is consumed only by code that opts into it. Ten unit tests, written test-first, pin the disentanglement (rule_3 synonyms group; rule_1/rule_2 objects separate), the frozen-query preservation, and that `reworded_query` only ever returns a same-concept synonym.

5.3 Results

The structural claim — that index 1 is a different *object* for rules 1–2 — is not just a naming argument; it is measurable in where Florence-2 puts the box. Grounding the primary phrase versus (a) a proposed same-concept synonym and (b) the frozen cross-concept phrase on 20 samples per rule, and measuring agreement with the primary term’s box (IoU; a missing or non-overlapping box scores 0):

rule	compared phrase	box agreement	grounded
rule_1	<i>synonym</i> safety helmet	0.281	20/20
rule_1	<i>cross</i> high-visibility vest	0.000	18/20
rule_2	<i>synonym</i> fall-arrest harness	0.426	13/20
rule_2	<i>cross</i> fall-protection lanyard	0.000	13/20

Table 5.2: Grounding agreement with the primary concept phrase. The cross-concept phrases DO ground a box (18/20, 13/20) — but at a systematically different location: agreement is exactly 0.000. True synonyms ground the same region (non-zero agreement).

The result is unusually clean. The cross-concept phrases are not failing to detect anything — “high-visibility vest” finds a box in 18 of 20 rule_1 images, “fall-protection lanyard” in 13 of 20 rule_2 images. But those boxes sit on a *different object*, so their overlap with the hard-hat / harness box is exactly zero. Swapping index 0 for index 1, as the pilot’s rewording test did, therefore relocated the grounding to a different object every time it grounded at all — which is why any answer or box change it produced cannot be attributed to wording. The true synonyms, by contrast, land on the same object (agreement 0.281 and 0.426): imperfect box overlap, as expected from Florence-2’s phrasing-dependent box jitter, but unambiguously the *same region*, not a different one.

5.4 An Honest Note on Synonym Strength

The same-concept agreements (0.281, 0.426) are modest, not near-unity, and that is worth stating plainly rather than dressing up. Two effects are mixed in: Florence-2 genuinely returns somewhat different box extents for different phrasings of the same object, and the proposed synonyms (“safety helmet,” “fall-arrest harness”) are candidates, not yet blessed. The principled definition the structure adopts is exactly this measurement: a proposed synonym is *validated* when it grounds substantially the same region as the primary phrase. By that definition “safety helmet” and “fall-arrest harness” are weakly validated — clearly the same object (non-zero, versus the cross term’s exact zero), but with enough box jitter that a scale run should either confirm them on more samples or prefer tighter synonyms. The structure makes that a checkable property rather than an assumption; the full study will run this check across the whole set before committing a synonym list.

5.5 Standard vs. Non-Standard Choices

Standard. Separating a controlled variable (wording) from a confound (object identity) is basic experimental design; a robustness-to-rewording test that swaps objects is simply mis-specified.

Non-standard, and the contribution. Two choices. First, “validated synonym” is given an operational, falsifiable definition — same-region grounding, measured — rather than left to intuition, so the query dictionary can be audited empirically instead of argued about. Second, the frozen queries and the entire pilot result set are preserved untouched beside the new structure, so the mis-specified Day-9 rule_1/rule_2 numbers remain reproducible as exactly what they were, and the corrected rewording test is a new, opt-in path rather than a silent redefinition of the old one.

5.6 Implications

The rewording-robustness test can now be run as a genuine phrasing-sensitivity measurement, swapping only validated same-concept synonyms, with the corrected result reported separately from the frozen (mis-specified) one. The concept structure also feeds two later needs: rule_1’s two-concept, “all”-satisfaction semantics is the hook for multi-object PPE checking (does a worker have both hard hat and vest?), and the per-concept synonym lists give the native-VQA models a clean vocabulary to be prompted with. The immediate carry-forward is a full-set synonym-validation pass to promote the weakly-validated candidates or replace them.

Chapter 6

Unifying Decoding to a Single Greedy Path (Phase 1.6)

6.1 Goal

The pilot evaluated its metrics across three different decoding regimes without treating that as a variable. Baseline inference, descriptive-accuracy masking, and robustness all used beam search (`num_beams=3`); attribution used greedy decoding (`num_beams=1`); stability used sampling (`do_sample=True`). Because decoding is not a neutral implementation detail — it changes which token sequence, and therefore which grounded box, the model emits — metrics computed on different decoding paths are not strictly comparable, and, worse, an explanation computed on one path can be explaining a different answer than the baseline reported on another.

The plan (item 3.6) prescribes adopting greedy decoding as the single canonical deterministic path, for two stated reasons: it is the path attribution can be extracted from without beam-reordering bookkeeping, and the pilot’s own efficiency profiling suggested beam and greedy cost about the same for short grounding outputs, so nothing is lost by switching. Stability’s sampling stays (that stochasticity is the metric’s point) but is anchored against the greedy path.

6.2 What Was Done

The unification is centralized rather than scattered. `config.DECODING` is a single switch ("`beam`" → 3 beams, the frozen default; "`greedy`" → 1), and `model.run_task` now resolves its beam count from that switch when no explicit value is passed (`_effective_num_beams`). Because every deterministic grounding call — baseline, masking, robustness, sparsity’s region lookups — flows through `run_task` without specifying beams, flipping one config value moves all of them onto greedy together. The two paths that legitimately differ pass their beam count explicitly and are therefore untouched: stability (`do_sample=True`, `num_beams=1`) and attribution (its own greedy `generate`). Sparsity was already greedy, since it reads cross-attention from the attribution path.

The default is "`beam`", so the frozen pilot reproduces byte-for-byte; scripts 03, 05, and 09 expose a `--decoding` flag, and any non-default run writes to a `greedy`-suffixed file. Four unit tests pin the resolver: explicit values always win, and `None` resolves from the current policy.

6.3 Results

Re-running the full 163-sample baseline under greedy and comparing to the frozen beam baseline:

measure	beam vs. greedy
answer agreement (overall)	93.3% (11 of 163 differ)
rule_1 / rule_2 / rule_3 / rule_4	92.1% / 92.3% / 93.9% / 96.0%
mean inference time	271 ms (beam) vs. 261 ms (greedy), ratio 0.96
median inference time	261 ms vs. 250 ms

Table 6.1: Greedy vs. beam baseline over 163 samples. Greedy costs essentially the same ($0.96\times$), confirming the efficiency rationale, and agrees with beam on 93.3% of answers.

Both plan claims hold. Greedy is not more expensive — it is marginally cheaper ($0.96\times$), consistent with the pilot’s observation that beam search buys little for outputs this short. And the answer agreement is high, 93.3%, so unifying onto greedy does not upend the pilot’s conclusions.

The more important number is the 6.7% that *disagree*, because it quantifies a fragmentation the pilot could not see. Under the old regime, the baseline reported a beam-search answer while the attribution (sparsity, heatmaps) was computed under greedy. For roughly one sample in fifteen, those two paths produce different grounded boxes and therefore potentially different answers — meaning the pilot’s own explanation for that sample was, strictly, explaining a decoding path the baseline never reported. Unifying the deterministic path removes that mismatch by construction: after the switch, the answer a metric reports and the answer its attribution explains are produced by the same decoder. The 93.3% agreement is thus the measure of how much the pilot’s cross-metric decoding inconsistency mattered — modest in aggregate, real at the per-sample level, and now eliminated.

The frozen `baseline_predictions.csv` is untouched (empty `git diff`); the greedy run wrote `baseline_predictions_greedy.csv`. The full suite is green at 109 tests.

6.4 Standard vs. Non-Standard Choices

Standard. Fixing one decoding configuration across an evaluation, rather than letting each metric inherit whatever default its code happened to use, is basic experimental control.

Non-standard, and the contribution. Two things. The unification is a *single* switch read at call time, so there is exactly one place decoding is decided and no metric can silently drift onto a different path — and the switch defaults to the frozen policy, so the pilot’s beam-search numbers remain the reproducible baseline. And the choice of greedy is justified by evidence rather than convention: it was adopted specifically because it is the path attribution can be read from, and only after confirming — here, at $0.96\times$ — that it costs no more than the beam search it replaces. The plan flags one caveat this validates conditionally: if a later decision adopts dense captioning (long outputs), the beam-vs-greedy cost equivalence must be re-measured, because the decode loop stops being nearly free.

6.5 Implications

Attribution (Phase 4) can now be read from the same decoder that produces the reported answer, so a saliency map or cross-attention heatmap explains the answer the baseline actually gave — a precondition for the two-stream attribution-agreement work. Every deterministic metric shares one

execution path, so cross-metric comparisons are no longer confounded by decoding. And the switch is one config line for the scale run, with the frozen beam numbers preserved beside the greedy ones for reference.

Chapter 7

Hardcoded-Assumption Audit and a CI Guard (Phase 1.7)

7.1 Goal

Chapter 13 of the pilot report found a real bug the hard way: `11_make_figures.py` had the sample count (“163”) and per-class sizes typed as string literals into its chart labels, so the moment the script ran on a different sample size it produced a chart confidently mislabeled with the wrong numbers. That bug survived twelve days and a passing 61-test suite precisely because every run until then happened to use the one sample size for which the literals were correct. The scale-up plan (item 3.7) generalizes the lesson: sweep every analysis and plotting path for literal sample counts, class sizes, grid dimensions, and the patch-grid arithmetic (the $768 \rightarrow 24$ image-token math), replace each with a value read from data or config, and add an automated check that fails if a known- magic literal ever reappears. This is the phase that designs the bug *class* out, not just the one instance Chapter 13 caught.

7.2 What Was Done

7.2.1 The patch-grid arithmetic, now derived

The clearest remaining instance was in `attribution.py`, which hardcoded the cross-attention heatmap grid as `PATCH_GRID = (24, 24)` with `N_PATCH_TOKENS = 576` — values correct only for Florence-2’s fixed 768×768 processor input. These are now computed at run time from the *actual pixel values* resolution and the vision tower’s documented stride (`_patch_grid`), and the reshape is protected by an assertion that the encoder sequence is long enough to hold the global token plus every patch token — so a layout change fails loudly instead of silently shifting every heatmap by a cell. The one retained constant, `VISION_TOWER_STRIDE = 32`, is an architectural fact (the DaViT tower’s $4 \times 2 \times 2 \times 2$ downsampling), not a resolution assumption. Verified directly: `_patch_grid` returns `(24, 24)` for a 768×768 input — reproducing the frozen grid exactly — and `(32, 24)` for a 1024×768 input, adapting as intended.

7.2.2 The remaining label literals

Two output labels still carried hardcoded counts and were switched to data-derived values: `08_efficiency_test.py`’s plot title “...(163-sample pilot)” now interpolates the observed sample count it already computes, and `06_visual_sparsity.py`’s “.../ 576 cells” summary now reads the cell count from the actual heatmaps (recorded as an `n_cells` column) rather than assuming 576. Model IDs were checked and are already sourced from `config.MODEL_ID` everywhere they drive behavior; only cosmetic “Loading Florence-2...” log strings mention the name literally, which affects nothing computed.

7.2.3 A CI guard that tells code from documentation

The automated check is a test, `test_no_hardcoded_literals.py`, that tokenizes every file under `scripts/` and `src/xai_pilot/` and fails if a forbidden numeric literal — `{163, 3004, 576, 768}`: the pilot/test sample counts and the patch-grid dimensions — appears as an actual `NUMBER` token in code. Using the tokenizer rather than a text grep is the point: comments and docstrings that legitimately *describe* the pilot’s 163 samples or explain why $576 = 24 \times 24$ are not `NUMBER` tokens and are correctly ignored, so the guard catches a real regression (a literal typed into an expression) without firing on prose that mentions the same number. The check runs as part of the ordinary `pytest` suite, so it gates every change the way any other test does.

7.3 Results

check	outcome
<code>_patch_grid(768×768)</code>	(24, 24) — frozen grid reproduced
<code>_patch_grid(1024×768)</code>	(32, 24) — adapts to input resolution
CI literal guard	passes (no forbidden literal in code)
full test suite	110 passed

Table 7.1: Phase 1.7 verification. The patch grid reproduces the pilot’s grid for the pilot’s input and adapts for a different one; the CI guard is active and green.

The audit’s effect is preventative rather than a headline number: the attribution heatmap now cannot misalign if a scale-run model uses a different input resolution, the two remaining mislabel risks are closed, and any future reintroduction of a dataset count or patch dimension as a code literal fails the suite immediately — the same class of bug Chapter 13 found by accident is now caught by construction.

7.4 Standard vs. Non-Standard Choices

Standard. Reading counts from data instead of hardcoding them, and asserting invariants at the point they are relied upon, are ordinary defensive-programming practice.

Non-standard, and the contribution. Two choices. The patch grid is not merely moved to a constant but *derived from the actual input and guarded by an assertion*, so the code is correct by construction across input sizes rather than correct by coincidence at one size — directly answering the plan’s instruction to recompute the arithmetic rather than hardcode it. And the CI guard is deliberately tokenizer-based so it can distinguish a magic literal in *code* from the same number appearing in a comment or docstring; a naive grep would either miss real regressions or drown in false positives on the many places the report’s prose legitimately cites 163 or 576. Encoding “a literal here is a bug, the same number in a sentence is documentation” into the check is what makes it maintainable enough to actually keep passing.

7.5 Implications

With the patch grid derived and asserted, Phase 4’s attribution work and Phase 5’s additional models can run at whatever input resolution they require without a silent heatmap-misalignment risk. The CI

guard makes the “every number traces to data or config” principle enforceable rather than aspirational, which matters most exactly when the codebase grows during the scale run and manual vigilance is least reliable. This closes Phase 1: the six foundational fixes and this audit together leave a pipeline whose metrics test the right region, measure model faithfulness rather than proxy plumbing where the fixes reach, and reproduce the frozen pilot byte-for-byte under their default flags.